

1 Documentation for the Profiler Output

The output of the profiler is a stream of Json objects with each line having a specific type in one of the below formats

1.1 Configuration Lines

```
{"Type": "_", "Version":1, "IsCover": false, "TimeType": "WallTime"}
```

At the start of each of the profiler files, there is one line in the format above, labelled "Type": "_". This type of line appears once per parse and specifies the global parameters of the profile. - Version : the version of the profiler - IsCover : specifies whether the profile given is for coverage or for timing. The coverage case is ignored in the existing profile visualization - TimeType : the type of time can be CPUTime, WallTime or Memory which is the memory allocated rather than time taken by a given line/function. - Note that Memory is treated like time in both the GAP profiler and this module, hence there are no memory-specific features.

1.2 File Registration Lines

```
{"Type": "S", "File": "path", "FileId": 262}
```

Lines labelled "Type": "S" are for file registration. Each of these lines assigns a file a unique ID so that when the files are referenced in the future they can be referenced by unique integers and not their full path's which could be long. - File : the path to the file - FileId the path's unique integer Id by which it will be referenced in the future

1.3 Read and Execute Lines

```
{"Type": "R", "Ticks": 83, "Line": 3, "FileId": 1304}
```

The lines labelled "Type": "R" specify how long the given line took to parse and interpret (via the GAP interpreter). Similarly, the lines labelled "Type": "E" specify how long the line took to execute after it had been interpreted. - Ticks : the amount of time taken to execute/interpret the line (in wall time or cpu time depending on the configuration) - Line : the line of the file which the code executed is on - FileId : the unique integer id of the file the executed code is contained in as specified by a line of type S earlier in the file

1.4 Enter Function Lines

```
{"Type": "I", "Fun": "funcname", "Line": 10, "EndLine": 14, "File": "path", "FileId": 1304}
```

When a function is called, and hence entered into, it is represented by a line labelled "Type": "I". These lines do not correspond to any actual code being executed, but rather specify that some scope is being entered or has been exited.

- Fun : name of the function as a string - Line : start line of the function -
EndLine : the line the function ends on - File : relative path from the current
directory to the file the function is contained in - FileId : the unique integer id
of the file the function is contained in

1.5 Exit Function Lines

```
{"Type":"0","Fun":"funcname","Line":344,"EndLine":351,"File":"path","FileId":23}
```

When a function is exited (in the normal way, not via an error) it is represented by a line labelled "Type": "0". These lines are almost identical to the I lines, and their fields mean the same things. One notable detail here is that when an error occurs in GAP, it forces exiting of all scopes until the error is handled (if not the code exits). In this output format this is represented by "Type": "0" lines, with their function names being **nameless**. These lines do not provide much data, and are there more to make sure the entity reading this output format can make sure it knows a scope was exited.